Gráfalgoritmusok 5
Legrövidebb utak súlyozott gráfokban

Szerző: Horváth Gyula
horvath@inf.elte.hu

2. Súlyozott gráfok ábrázolása

```
#include <bits/stdc++.h>
#define maxN 1001 //pontok max száma
using namespace std;

typedef
  int Graf1[maxN][maxN]; //szomszédsági mátrix
struct
  El{
    int pont;//az él oda pontja
    int suly;//az élek súlya
  };
//
typedef
  vector<El> Graf2[maxN];
//
struct SEl{
  int p,q;//p->q él
  int suly;
};
typedef
  vector<SEl> Graf3[maxN];
```

3. Legrövidebb utak minden pontpárra

Bemenet: $G = (V, E, s)$ súlyozott (irányított vagy irányítatlan) gráf, $s : E \to \mathbb{R}^+$.
Kimenet: Legrövidebb utak $T(i, j)$ értéke minden $i, j \in V$ pontpárra, $Elso(i, j)$ egy legrövidebb út meghatározására.
Feltesszük, hogy $V = \{1, \ldots n\}$
Jelölje $K(u)$ az $u = (p_0, \ldots, p_m)$ út hosszát (költségét), az útban lévő élek súlyainak összegét: tehát

$$K(u) = \sum_{i=0}^{i<m} s(p_i, p_{i+1})$$

Jelölje $\delta(i, j)$ a legrövidebb út hossza, azaz

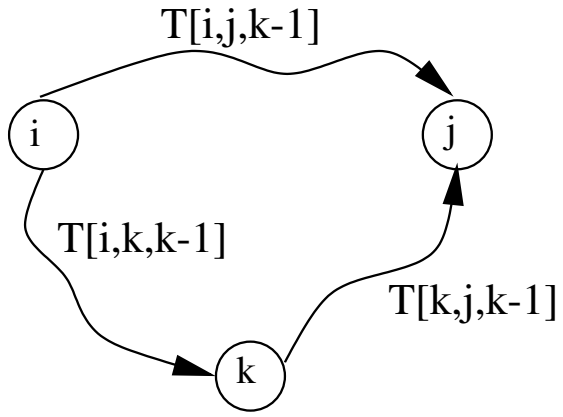
$$\delta(i, j)) = \begin{cases} \infty \text{ ha nincs út } i\text{-ből } j - \text{be} \\ \min\{K(u) : u : i \rightsquigarrow j\} \text{ egyébként} \end{cases} \tag{1}$$

$$\delta(i, j) = \min\{K(u) : u : i \rightsquigarrow j\}$$

A megoldás előállítására alkalmazzuk a dinamikus programozás módszerét: részproblémákra bontás.
Minden $i, j \leq n$ és $0 \leq k \leq n$ esetére számítsuk ki a $T[i, j, k]$ értékeket a kövező rekurzív összefüggés alapján:

$$T[i, j, 0) = \begin{cases} 0 & \text{ha } i = j \\ \infty & \text{ha nincs } (i, j) \notin E \\ s(i, j) & \text{ha } (i, j) \in E \end{cases} \tag{2}$$

$$T[i, j, k] = \min(T[i, j, k-1], T[i, k, k-1] + T[k, j, k-1]) \quad \text{ha} \quad k > 0 \tag{3}$$



1. ábra

Az (1) és (2) összefüggések alapján minden $T[i, j, k]$ értékhez, ha az nem ∞, megadhatunk egy $U(i, j, k) : i \rightsquigarrow j$ utat, amelyek hossza $T[i, j, k]$ és minden belső p pontjára teljesül, hogy $p < k$.
Ha (2) szerint $T[i, j, k] = T[i, k, k-1] + T[k, j, k-1]$, akkor feltehető, hogy az $U(i, k, k, 1)$ és az $U(k, j, k-1)$ utaknak csak a k közös pontja. Ha ugyanis p közös pontjuk lennek, akkor $i \rightsquigarrow p \rightsquigarrow k \rightsquigarrow p \rightsquigarrow j$ pontsorozatban $p \rightsquigarrow k \rightsquigarrow p$ részt p-vel helyettesítve olyan utat (pontsorozatot) kapnánk, amely nem tartalmazza a k

pontot és hossza $\leq$ T[i, j, k]. Tehát U(i, k, k − 1) és az U(k, j, k − 1) egymás utáni alkalmazásával olyan utat kapunk, amelynek minden belső p pontjára p < k.

Jelölje δ(i, j, k) (k = 0, 1 . . . , n) az olyan i-ből j-be vezető legrövidebb út hosszát, amelynek minden belső p pontjára teljesül, hogy p $\leq$ k. Tehát

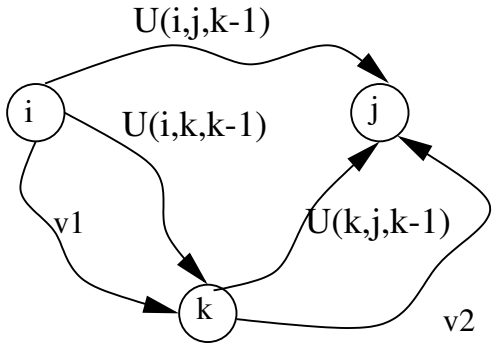
$$\delta(i, j) = \delta(i, j, n)$$

Megmutatjuk, hogy az algoritmus által kiszámított T(i, j, k) értékekre teljesül, hogy

$$\delta(i, j, k) = T(i, j, k)$$

A bizonyítást k értéke szerinti teljes indukcióval végezzük.
Ha nincs i-ből j-be út, akkor minden k-ra T[i, j, k] = ∞ = δ(i, j)
Tegyük fel, hogy T[i, j, k] $\neq \infty$
Ha k = 0, azaz az úton nincs közbülső pont, akkor
δ(i, j, 0) = s(u, v) = T[i, j, 0]
Tegyük fel, hogy k > 0 és minden l < k-ra teljesül, hogy

$$\delta(i, j, l) = T[i, j, l] = K(U(i, j, l))$$



2. ábra

Legyen v = (i, . . . , j) egy legrövidebb út i-ből j-be, amelynek minden p belső pontjára p $\leq$ k, tehát K(v) = δ(i, j, k).

Ha a v út nem tartalmazza a k pontot,
akkor az indukciós feltevés szerint δ(i, j, k) = δ(i, j, k − 1) = T(i, j, k).
Tegyük fel, hogy a v út tartalmazza a k pontot. Ekkor v megadható a v1 : i $\rightsquigarrow$ k és v2 : k $\rightsquigarrow$ j utak egymás utáni alkalmazására.
Mivel minden legrövidebb út minden részútja is legrövidebb, és nem tartalmazzák belső pontként a k pontot ezért

$$K(v1) = \delta(i, k, k-1) \text{ és } K(v2) = \delta(k, j, k-1)$$

Mivel az indukciós feltevés szerint U(i, k, k − 1) és (U(k, j, k − 1) legrövidebb olyan út, amelynek minden belső pontja < k , ezért

$$\begin{aligned}
\delta(i, j, k) &= K(v1) + K(v2) && (4)\\
&= \delta(i, k, k-1) + \delta(k, j, k-1) && (5)\\
&= K(U(i, k, k-1)) + K(U(k, j, k-1)) && (6)\\
&= T[i, k, k-1] + T[k, j, k-1] = T[i, j, k] && (7)
\end{aligned}$$

A `T[i, j, k]` értékek tarolására elég egy `T[]` tömb, mert csak a `T[i, k, k−1]` és a `T[k, j, k−1]` íródik felül, amire a szükség lenne a `k` ciklusban, azonban `T[i, k, k] = T[i, k, k−1] + T[k, k, k−1] = T[i, k, k−1]`, mert `T[k, k, l] = 0` minden `l`-re.

3.1. Megvalósítás: Floyd-Warshall algoritmus

```
#include <iostream>
#define maxN 1001
#define Infi 10000000
using namespace std;

typedef
int Graf[maxN][maxN]; //szomszédsági mátrix
Graf T;
//long long T[maxN][maxN];
int Elso[maxN][maxN];

void Floyd_Warshall(Graf G, int n, Graf T){
  //Global:Elso
  for (int i=1;i<=n;i++)//inicializálás, T:=G
  for (int j=1;j<=n;j++){
    T[i][j]=G[i][j];
    if(G[i][j]!=Infi) Elso[i][j]=j; else Elso[i][j]=0;
  }
  for (int k=1;k<=n;k++)
  for (int i=1;i<=n;i++)
  for (int j=1;j<=n;j++){
    long m=(T[i][k]+T[k][j]);
    if (m<T[i][j]){
      T[i][j]=m;
      Elso[i][j]=Elso[i][k];
    }
  }
}
```

```
1  void UtKiIro(int i, int j){
2    //i~>j legrövidebb út kiíratása
3    //Global: Elso
4    do{
5      cout<<i<<" ";
6      i=Elso[i][j];
7    }while(i!=j);
8    cout<<j<<endl;
9  }
10 int main(){
11   int n, m,p,q,s;
12   Graf G;
13   cin>>n>>m;
14   //inicializálás
15   for(int p=1;p<=n;p++){
16     //G[p][p]=0; //ha nem lehet hurokél
17     for(int q=p+1;q<=n;q++){
18       G[p][q]=Infi;
19       //G[q][p]=Infi; //ha irányítatlan
20     }
21   }
22
23   for(int i=0;i<m;i++){
24     cin>>p>>q>>s;
25     G[p][q]=s;
26     //G[q][p]=s; //ha irányítatlan
27   }
28
29   Floyd_Warshall(G, n, T);
30   int A,B;
31   cin>>A>>B;
32   cout<<T[A][B]<<endl;
33   if(T[A][B]!=Infi) UtKiIro(A,B);
34 }
```

A Floyd-Warshall algoritmus futási ideje legjobb és legrosszabb esetben is $\mathbb{O}(n^3)$.

4. Feladat: Legnagyobb átviteli sebesség

Minden számítógépes hálózat csomópontokból és bizonyos csomópontpárok között kiépített közvetlen kétirányú adatátvitelt biztosító kommunikációs vonalakból épül fel. Minden közvetlen vonalnak adott átviteli sebességgel tud kommunikálni. Ha egy `P` csomópontból egy `Q` csomópontba kell átvitelt megvalósítani amely egy `U` útvonalon történik, akkor a tényleges átviteli sebesség az útvonalban lévő közvetlen vonalak átviteli sebességeinek a minimuma. Írj programot, amely megadott csomópont párokra kiszámítja a lehető legnagyobb átviteli sebességet, és megy adja a megfelelő útvonalakat!

Bemenet

Bemenet A standard bemenet első sorában három egész szám van, a csomópontok száma ($1 < N \leq 1\,000$), azon csomópontpárok száma ($1 < M \leq 100\,000$), amelyek között közvetlen adatátvitel van kiépítve, és a kérdések ($1 \leq K \leq N$) száma. A további M sor mindegyike három egész számot tartalmaz: $u\ v\ s$ ($1 \leq u \neq v \leq N$) ($0 \leq s \leq 1000$), amely azt jelenti, hogy az u és v csomópont között olyan közvetlen kétirányú adatátviteli vonal van kiépítve. amely maximális átviteli sebessége s. Bármely két csomópont között legfeljebb egy közvetlen vonal van.

Kimenet

Kimenet A standard kimenetre `K` sort kell írni. Az `i`-edik sor az `i`-edik kérdésre adandó válsz legyen. A sorban ez első szám a lehetséges legnagyobb átviteli sebesség legyen. A sorban ezt követően kell kiírni egy olyan útvonalat, amely a legnagyobb átviteli sebességet biztosítja. Több megoldás esetén bármelyik megadható. Ha nincs útvonal a kérdéses két csomópont között, akkor az egyetlen 0 számot kell kiírni.

Példa

Bemenet

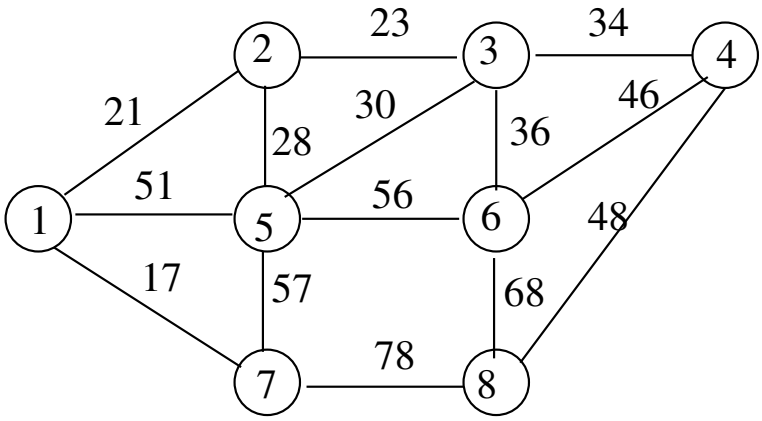
```
8 15 3
1 2 21
1 5 51
1 7 17
2 3 23
2 5 25
3 5 30
3 4 34
3 6 36
4 6 46
4 8 48
5 6 56
5 7 57
6 7 40
6 8 68
7 8 78
4 5
1 7
7 3
```

Kimenet

```
48 4 8 7 5
51 1 5 7
36 7 5 6 3
```

Pontozás

Pontozás A pontok `50%`-a minden tesztesetre, ha a kimenet első sora helyes.
A pontok `10%`-a kapható olyan tesztekre, ahol $N < 16$.



3. ábra. A legnagyobb átviteli sebesség `1` és `4` között az `1` → `5` → `7` → `8` → `4` útvonalon 48.

A pontok `30%`-a kapható olyan tesztekre, ahol a megoldás elemszáma 1.
A pontok `60%`-a kapható olyan tesztekre, ahol nincs egyéb korlátozás.

Korlátok

Korlátok $1 < N \leq 1\,000$
$1 \leq M \leq 100\,000$
$1 \leq K \leq 1\,000$
Időlimit: `0.1` másodperc
Memórialimit: `32` MB.

4.1. Általános optimális útprobléma

Az a sejtésünk, hogy a legnagyobb átvitel probléma megoldható a Floyd-Warshall algoritmus olyan módosításával, hogy a + összeadás helyett a `min` műveletet, a `min` reláció helyett pedig a `max` relációt alkalmazzuk.
A megoldás helyességét úgy igazoljuk, hogy legrövidebb út problémát általánosabban fogalmazzuk meg.
Legyen $G = (V, E, s)$ súlyozott (címkézett) gráf, ahol $s : E \to S$ súlyozó (címkéző) függvény.
Minden $u : a \rightsquigarrow b = (a = p_0, p_1, \ldots, p_m = b)$ út költségét egy $\oplus : S \times S \to S$ kétváltozós művelttel adjuk meg, jele $\bigoplus(u)$. Ezt a függvényt a probléma **költségfüggvényének** nevezzük.

$$\bigoplus(u) = s(p_0, p_1) \oplus \cdots \oplus s(p_{m-1}, p_m)$$

Olyan utat keresünk minden $a, b \in V$ pontpárra, amelynek a költsége optimális. Az optimalitást egy adott $\trianglelefteq : S \times S \to \{0, 1\}$ lineáris (teljes) rendezési relációval adjuk meg:
Az $u : a \rightsquigarrow b$ **optimális költségű** az (a, b) pontpárra, ha bármely $v : a \rightsquigarrow b$ út esetén

$$\bigoplus(u) \trianglelefteq \bigoplus(v)$$

A $\trianglelefteq$ rendezési relációt a probléma **optimalizáló** rendezésének nevezzük. Ilyen lehet például a `min` és a `max` rendezési reláció.

Legyen $\oplus$ költségfüggvény és $\trianglelefteq$ optimalizáló bináris reláció. Minden i, j pontpárra határozzuk meg azt a $\delta(i, j)$ értéket, amelyre teljesül, hogy minden $u : i \rightsquigarrow j$ útra teljesül, hogy:

$$\delta(i, j) \trianglelefteq \bigoplus(u)$$

Ha $\delta(i, j) = \bigoplus(u)$, akkor azt mondjuk, hogy az $u : i \rightsquigarrow j$ út optimális.
Megmutatjuk, hogy az általános optimális útprobléma milyen feltételek teljesülése esetén oldható meg a Floyd-Warshall algoritmus elve alapján, azaz ha összeadás helyett $\oplus$ és `min` helyett $\trianglelefteq$ műveleteket veszünk.
Számítsuk ki minden i, j és $k < leqn$ -re a $\oplus$ költségfüggvénnyel és a $\trianglelefteq$ lineáris rendezési relációval a $T[i, j, k]$ költség értékeket.
Feltételezzük, hogy létezik két kitüntetett súly érték, a $\infty \in S$ és a $0 \in S$, így minden pontpárra, nem csak az élekre értelmezett az $\oplus : S \times S \to S$ költségfüggvény és a $\trianglelefteq$ rendezési reláció.

$$T[i, j, 0) = \begin{cases} 0 & \text{ha } i = j \\ \infty & \text{ha nincs } (i, j) \notin E \\ s(i, j) & \text{ha } (i, j) \in E \end{cases} \tag{8}$$

$$T[i, j, k) = \begin{cases} T[i, k, k-1] \oplus T[k, j, k-1] \\ \quad \text{ha } T[i, k, k-1] \oplus T[k, j, k-1] \trianglelefteq T[i, j, k-1] \\ \quad \text{és } T[i, k, k-1] \oplus T[k, j, k-1] \neq T[i, j, k-1] \\ \\ T[i, j, k-1] \text{ egyébként} \end{cases} \tag{9}$$

Milyen tulajdonsággal kell rendelkeznie a $\oplus$ **költségfüggvénynek** és a $\trianglelefteq$ **optimalizáló** rendezési relációnak ahhoz, hogy a Floyd-Warshall algoritmus optimális megoldást állítson elő?

A fenti kiszámítás optimális megoldást ad, ha minden $u : i \rightsquigarrow j$ útra teljesül, hogy

$$T[i, j, n] \trianglelefteq \bigoplus(u)$$

4.2. Optimalitási feltételek a $\oplus$ költségfüggvényre és a $\trianglelefteq$ optimalizáló rendezésre

$\oplus$ **asszociatív:** $(x \oplus y) \oplus z = x \oplus (y \oplus z)$

$0 \in S$ **a** $\oplus$ **egységeleme:** $x \oplus 0 = 0 \oplus x = x$

$\infty \in S$ **a** $\oplus$ **művelet elnyelője:** $x \oplus \infty = \infty \oplus x = \infty$

$\trianglelefteq$ **reflexiv:** $x \trianglelefteq x$

$\trianglelefteq$ **tranzitív:** ha $(x \trianglelefteq y)$ és $(y \trianglelefteq z)$ akkor $x \trianglelefteq z)$

$\trianglelefteq$ **antiszimmetrikus:** $(x \trianglelefteq y)$ és $(y \trianglelefteq x)$ akkor $x = y$

$\trianglelefteq$ **lineáris:** $x \trianglelefteq y$ vagy $y \trianglelefteq x$

$\oplus$ **disztributív** $\trianglelefteq$ **felett:** ha $x \trianglelefteq y$ igaz, akkor $(x \oplus z) \trianglelefteq (y \oplus z)$ igaz és $(z \oplus x) \trianglelefteq (z \oplus y)$ igaz

$\oplus$ **monoton a** $\trianglelefteq$ **renezésre:** $x \trianglelefteq x \oplus y$

Állítás: A monotonitásból és ∞ elnyelő tulajdonságából következik, hogy $x \trianglelefteq \infty$.

$$x \trianglelefteq (x \oplus \infty) = \infty$$

Fontos megjegyezni, hogy a feltételek azt jelentik, hogy az optimalizáló $\trianglelefteq$ reláció lineáris (teljes) rendezés az súlyok vagy értékek S halmazán.

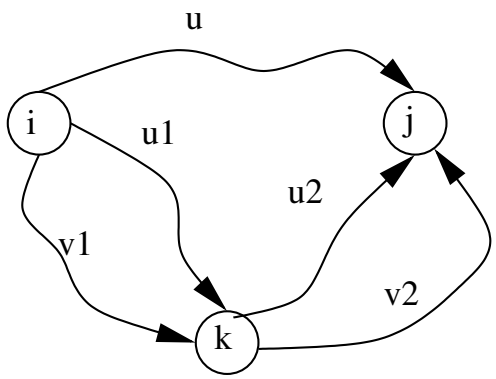
4.1. tétel. *Ha a $\oplus$ és $\trianglelefteq$ műveletekre teljesül az optimalitási feltétel, akkor az általános Floyd-Warshall algoritmus optimális megoldást ad.*

Bizonyítás

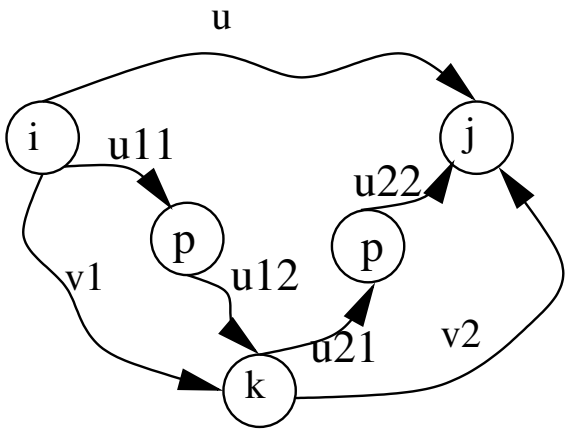
Jelölje $\delta(i, j)$ az optimális $i \rightsquigarrow j$ út költségét, $\delta(i, j, k)$ pedig az olyan $i \rightsquigarrow j$ utak optimális költségét, amelyre teljesül, hogy minden p belső pontjára $p <= k$.
A bizonyítást k-szerinti teljes indukcióval végezzük.
A $k = 0$ eset:
Ha nincs $i \to j$ él a gráfban, akkor $\delta(i, j, 0) = T(i, j, 0) = \infty$,
egyébként $\delta(i, j, 0) = T(i, j, 0) = s(i, j)$.
A $k > 0$ eset:

Tegyük fel, hogy minden $l < k$ esetén $T(i, j, l) = \delta(i, j, l)$.
Ha $T[i, j, k-1] \trianglelefteq T[i, k, k-1] \oplus T[k, j, k-1]$, akkor az indukciós feltevés miatt $\delta(i, j, k) = T[i, j, k-1]$.
Tegyük fel, hogy $T[i, k, k-1] \oplus T[k, j, k-1] \trianglelefteq T[i, j-k-1]$
és $T[i, k, k-1] \neq T[k, j, k-1] \trianglelefteq T[i, j-k-1]$
Tehát $T[i, j, k] = T[i, k, k-1] \oplus T[k, j, k-1]$ lesz.
Legyen $u1 : i \rightsquigarrow k$ és $u2 : k \rightsquigarrow j$ az algoritmus által előállított olyan út, hogy $\bigoplus(u1) = T[i, k, k-1]$ és $\bigoplus(u2) = T[k, j, k-1]$.
Feltehetjük, hogy $u1$ és $u2$ nem tartalmaz közös belső pontot. Ha p közös belső pontjuk lenne, akkor az $u1 \circ u2$ utat bontsuk fel p szerint:

$$u1 \circ u2 = u11 \circ u12 \circ u21 \circ u22 = i \rightsquigarrow p \rightsquigarrow k \rightsquigarrow p \rightsquigarrow j$$



4. ábra



5. ábra

A monotonitás miatt

$$\bigoplus(\mathtt{u11}) \trianglelefteq \bigoplus(\mathtt{u11}) \oplus \bigoplus(\mathtt{u12}) \oplus \bigoplus(\mathtt{u21})$$

A disztributivitást alkalmazva kapjuk, hogy

$$\bigoplus(\mathtt{u11}) \oplus \bigoplus(\mathtt{u22}) \trianglelefteq \bigoplus(\mathtt{u11}) \oplus \bigoplus(\mathtt{u12}) \oplus \bigoplus(\mathtt{u21}) \oplus \bigoplus(\mathtt{u22}) = \mathtt{T[i,\ j,\ k]}$$

Az $\mathtt{u11} \circ \mathtt{u22} : \mathtt{i} \rightsquigarrow \mathtt{j}$ út nem tartalmazza a $\mathtt{k}$ pontot, ezért az indukciós feltevés szerint
$\mathtt{T[i,\ j,\ k-1]} \trianglelefteq \bigoplus(\mathtt{u11}) \oplus \bigoplus(\mathtt{u22})$, tehát
$\mathtt{T[i,\ j,\ k-1]} = \mathtt{T[i,\ j,\ k]}$,
ami ellentmond azon feltevésünknek, hogy $\mathtt{p}$ közös belső pontja az $\mathtt{u1}$ és az $\mathtt{u2}$ útnak.
Legyen $\mathtt{v} : \mathtt{i} \rightsquigarrow \mathtt{j}$ olyan optimális út, amelynek minden belső pontjára teljesül, hogy $\mathtt{p} \leq \mathtt{k}$. Tegyük fel, hogy $\mathtt{v}$ belső pontként tartalmazza a $\mathtt{k}$ pontot, azaz megadható $\mathtt{v} = \mathtt{v1} \circ \mathtt{v2}$, $\mathtt{v1} : \mathtt{i} \rightsquigarrow \mathtt{k}$, $\mathtt{v2} : \mathtt{k} \rightsquigarrow \mathtt{j}$. Ekkor sem $\mathtt{v1}$ sem $\mathtt{v2}$ nem tartalmazza a $\mathtt{k}$ pontot. Az indukciós feltevés szerint teljesül, hogy

$$\mathtt{T[i,\ k,\ k-1]} = \bigoplus(\mathtt{u1}) \trianglelefteq \bigoplus(\mathtt{v1}) \text{ és } \mathtt{T[k,\ j,\ k-1]} = \bigoplus(\mathtt{u2}) \trianglelefteq \bigoplus(\mathtt{v2}),$$

tehát

$$\mathtt{T[i,\ j,\ k]} = \bigoplus(\mathtt{u1}) \oplus \bigoplus(\mathtt{u2}) \trianglelefteq \bigoplus(\mathtt{v1}) \oplus \bigoplus(\mathtt{v2}) = \bigoplus(\mathtt{v}) = \delta(\mathtt{i,\ j,\ k})$$

Definíció szerint

$$\delta(\mathtt{i,\ j,\ k}) \trianglelefteq \bigoplus(\mathtt{u1}) \oplus \bigoplus(\mathtt{u2})$$

tehát mivel $\trianglelefteq$ antiszimmetrikus, ezért

$$\mathtt{T[i,\ j,\ k]} = \delta(\mathtt{i,\ j,\ k})$$

Ezzel bebizonyítottuk, hogy az általános Floyd-Warshall algoritmus optimális megoldást ad, ha a $\oplus$ költségfüggvényre és a $\trianglelefteq$ optimalizáló lineáris rendezési relációra teljesül az optimalitási feltétel.
A standard esethez hasonló módon belátható, hogy elegendő egy $\mathtt{T[]}$ tömb a $\mathtt{T[i,\ j,\ k]}$ értékek számításához.

4.2. következmény. *Az általános Floyd-Warshall algoritmus helyes eredményt ad a Legnagyobb átvitel feladat megoldására, ha a költségfüggvény* $\oplus := \mathtt{min}$*, az optimalizáló reláció* $\trianglelefteq := \mathtt{max}$*, továbbá* $\infty := 0$ *és* $0 := \infty$*, mivel ezekre teljesül az optimalitási feltétel.*

4.3. A Legnagyobb átvitel feladat megvalósítása

```
#include <bits/stdc++.h>
#define Infi INT_MAX/2
using namespace std;

typedef vector<vector<int>> Graf; //szomszédsági mátrix

void Floyd_Warshall(Graf& G, Graf& T, Graf& Elso){
  int n=G.size()-1;
  T.resize(n+1); Elso.resize(n+1);
  for(int i=1;i<=n;i++){
    T[i].resize(n+1); Elso[i].resize(n+1);
  }
  for (int i=1;i<=n;i++)//inicializálás, T:=G
    for (int j=1;j<=n;j++){
      T[i][j]=G[i][j];
      if(i!=j && G[i][j]!=Infi) Elso[i][j]=j; else Elso[i][j]=0;
    }
  for (int k=1;k<=n;k++){
    for (int i=1;i<=n;i++){
      for (int j=1;j<=n;j++){
        int m=(T[i][k]+T[k][j]);
        if (m<T[i][j]){
          T[i][j]=m;
          Elso[i][j]=Elso[i][k];
        }
      }
    }
  }
}

void UtKiIro(int i, int j, Graf& Elso){
  //i~>j legrövidebb út kiíratása
  do{
    cout<<i<<" ";
    i=Elso[i][j];
  }while(i!=j);
  cout<<j<<endl;
}
```

```
39
40 int main(){
41   int n,m,p,q,s;
42   Graf G;
43   Graf T;
44   Graf Elso;
45   cin>>n>>m;
46   //inicializálás
47   G.resize(n+1);
48   for(int i=1;i<=n;i++) G[i].resize(n+1);
49   for(int p=1;p<=n;p++){
50     G[p][p]=0; //nem lehet hurokél
51     for(int q=p+1;q<=n;q++){
52       G[p][q]=Infi;
53       G[q][p]=Infi;
54     }
55   }
56   //Beolvasás
57   for(int i=0;i<m;i++){
58     cin>>p>>q>>s;
59     G[p][q]=s;
60     G[q][p]=s; //ha irányítatlan
61   }
62   int A,B;
63   cin>>A>>B;
64   Floyd_Warshall(G, T, Elso);
65   cout<<A<<"->"<<B<<endl;
66   if(T[A][B]==Infi){cout<<"Nincs út"; return 0;}
67   cout<<"Tav="<<T[A][B]<<endl;
68   if(T[A][B]!=Infi) UtKiIro(A,B, Elso);
69 }
```

Az általános Floyd-Warshall algoritmussal nem csak olyan problémát tudunk megoldani, ahol a gráf éleinek súlya (címkéje) számérték. Tekintsük a következő problémát.
Egy repülőtársaság N repülőtér között üzemeltet járatokat. Pontosabban, M repülőtérpár között van közvetlen járata. Kétféle repülővel végzi a szolgáltatást; A és B, de minden közvetlen járatra csak az egyik típusú gépét használja. Ha az U reptérről kell a V reptérre kell szállítania, akkor olyan útvonalat választ, amely a legkevesebb közbülső reptéren megy keresztül. Azonban gazdaságossági okokból az A típusú gépét preferálja, ezért ha több legrövidebb út is van, akkor azt választja, amelyiken a legtöbb az A típusú gép.
Határozzuk meg minden U V repülőtérpárra a kívánatos útvonalat (ha van).

Megoldás:
Költségérték: (a, b) számpár; $a, b \in \mathbb{N}$, $a + b > 0$. Egy $U \rightsquigarrow V$ költsége (a, b), ahol az útban a számú A és b számú B közvetlen járat van.
$\oplus := (a_1, b_1) \oplus (a_2, b_2) = (a_1 + b_1, a_2 + b_2)$
$\trianglelefteq := (a_1, b_1) \trianglelefteq (a_2, b_2)$ akkor és csak akkor, ha $a_1 + b_1 < a_2 + b_2$ vagy $a_1 + b_1 = a_2 + b_2$ és $a_1 > b_1$
$\infty := (N + 1, N + 1)$
$0 := (0, 0)$
Nem nehéz belátni, hogy az optimalitási feltételek teljesülnek.

Tekintsük azt a feladatot, amelyben egy súlyozott $G = (V, E, s)$ irányított gráf éleinek súlya alfabetikus karakter; $s : E \to \mathtt{char}$. Határozzuk meg minden u, v pontpárra a legrövidebb utat, ha több is van, akkor azt, amelyhez tartozó címkesorozat lexikografikusan a legkisebb.
A megoldás megadható az általánosított Floyd-Warshall algoritmussal, ahol a költség értékek stingek, a $\oplus$ költség a konkatenáció, a $\trianglelefteq$ optimalizáló lineáris rendezési reláció pedig: $\trianglelefteq : s_1 \trianglelefteq s_2$ akkor és csak akkor, ha $|s_1| < |s_2|$ vagy $|s_1| = |s_2|$ és s_1 lexikografikusan megelőzi s_2-t.
$\infty = ”\#”$
$0 = ””$
Belátható, hogy teljesül a két műveletre teljesül az optimalitási feltétel.

Mutassuk meg, hogy ha a legrövidebb ut költsége az úthoz tartozó karaktersorozat és a legrövidebb a lexikografikusan legkisebb lenne, akkor az a feladat nem oldható meg az általános Floyd-Warshall algoritmussal. Mutassuk meg, hogy az optimalitási feltételek közül melyik nem teljesül!

Mutassuk meg, hogy a leghosszabb út minden pontpárra probléma nem oldható meg a Floyd-Warshall algoritmussal! Adjuk meg, hogy optimalitási feltételek közül melyik nem teljesül!

5. Tranzitív lezárt

5.1. definíció. *A* $G = (V, E)$ *irányított (irányítatlan) gráf* ***tranzitív lezártja*** *az a* $G^* = (V, E^*)$ *gráf, amelyre teljesül, hogy* $(a, b) \in E^*$ *akkor és csak akkor, ha van* $a \rightsquigarrow b$ *út a* G *gráfban.*

A megoldás megadható az általános Floyd-Warshall algoritmussal.
Minden (i, j) pontpárt élnek tekintünk, az él súly 1 (igaz), ha $(i, j) \in E$, egyébként 0 (hamis). A tranzitív lezártban minden $u : i \rightsquigarrow j$ költsége (értéke) 1, ha van $u : i \rightsquigarrow j$ út az eredeti gráfban, egyébként 0. Tehát az u út költsége az $\mathtt{és}$ logikai művelettel számítható.
Az optimalizáló lineáris reláció pedig az, amely $(0, 1)$ argumentumok esetén hamis, egyébként igaz,
A költség értékek a $\{0, 1\}$ logikai értékek.
A $\oplus$ költségfüggvény: $x \oplus y = x \mathtt{ és } y$
A $\trianglelefteq$ optimalizáló lineáris rendezési reláció: $x \trianglelefteq y = x \mathtt{ vagy } (\mathtt{nem}\ y)$
A 0 egységelem := 1 (igaz)
A ∞ := 0 (hamis)

5.1. Megvalósítás

```
typedef
bool Graf[maxN][maxN]; //szomszédsági mátrix

bool TG[maxN][maxN];   //a tranzitív lezárt
int Elso[maxN][maxN];

void TranzitivLezart(Graf G, int n ){
//Global:TG, Elso
  for (int i=1;i<=n;i++)//inicializálás, TG:=G
    for (int j=1;j<=n;j++){
      TG[i][j]=G[i][j];
      if(i!=j && G[i][j]) Elso[i][j]=j; else Elso[i][j]=0;
    }
  for (int k=1;k<=n;k++)
    for (int i=1;i<=n;i++)
      for (int j=1;j<=n;j++){
        if(TG[i][j]) continue;
        TG[i][j]=TG[i][k] && TG[k][j];
        if(TG[i][j])
          Elso[i][j]=Elso[i][k];
      }
}
```

6. Leghosszabb utak

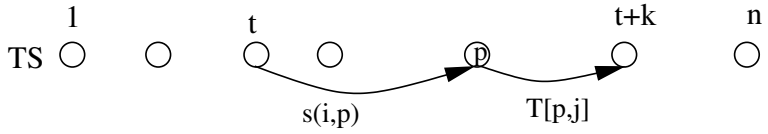
Súlyozott gráfok leghosszabb útjainak kiszámítására nem ismert hatékony (a pontok számában polinomiális idejű) algoritmus az általános esetre.
Körmentes gráfok esetén a Floyd-Warshall algoritmus elvéhez hasonló módszerrel $\mathbb{O}(n^3)$ futási idejű algoritmust tudunk adni.
Legyen $G = (V, E, s)$ súlyozott irányított, körmentes gráf, $s : E \to N^+$.
Számítsuk ki a pontok TS topologikus sorrendjét, és legyen iTS a TS permutáció inverze, tehát $TS[iTS[x]] = x$.
Nyilvánvaló, hogy csak olyan $i, j \in V$ pontikra létezik út, amelyekre $iTS[i] < iTS[j]$.
Az is igaz, hogy bármely $u : i \rightsquigarrow j$ út csak olyan p pontokon át haladhat, amelyre $iTS[i] < iTS[p] < iTS[j]$.
Vegyük észre, hogy minden $u : i \rightsquigarrow p \rightsquigarrow j$ leghosszabb út esetén a $p \rightsquigarrow j$ részút is leghosszabb (mivel a gráf körmentes és a súlyok nem-negatívak).
A részproblémákra bontás: $k = iTS[j] - iTS[i]$ szerint, $k = 1, \ldots n-1$ sorrendben számítsuk ki a legnagyobb $T[i, j]$ távolságot.
$k = 1$ esetén:

$$T[TS[i], TS[i+1] = \begin{cases} 0 & \text{ha } i = j \\ 0 & \text{ha } (TS[i], TS[i+1]) \notin E \\ s(TS[i], TS[i+1]) & \text{ha } (TS[i], TS[i+1]) \in E \end{cases} \tag{10}$$

$i = TS[t]$ és $j = TS[t+k]$, $k = 1$, $t + k \leq n$ és $k > 1$ esetén

$$T[i, j] = \max\{s(i, p) + T[p, j] : (i, p) \in E) \text{ és } iTS[p] \leq t + k\} \tag{11}$$

k-szerinti teljes indukcióval bizonyítható, hogy minden i, j párra, ha $iTS[i] < iTS[j]$, akkor $T[i, j]$ a leghosszabb $u : i \rightsquigarrow j$ út hossza.



6. ábra. $i = TS[t]$, $j = TS[t+k]$, $iTS[p] \leq t + k$

```
1  #include <bits/stdc++.h>
2  #define maxN 1001
3  #define Infi 10000000
4  using namespace std;
5  struct El{
6   int pont, suly;
7   El(int u, int s){
8     pont=u;
9     suly=s;
10  };
11 };
12 vector<El> G[maxN]; //szomszédsági lista
13
14 long long T[maxN][maxN];
15 int Elso[maxN][maxN];
16 int TS[maxN]; //topologikus sorrend
17 int iTS[maxN];//TS inverze: TS[iTS[x]]=x
18 int nTS;
19 bool Volt[maxN];
20 void MelyBejar(int p){
21  Volt[p]=true;
22  for(auto q:G[p]){
23    if(Volt[q.pont]) continue;
24    MelyBejar(q.pont);
25  }
26  iTS[p]=nTS;
27  TS[nTS--]=p;
28 }
29
30 void UtKiIro(int i, int j){
31  //i~>j legrövidebb út kiíratása
32  //Global: Elso
33  do{
34    cout<<i<<" ";
35    i=Elso[i][j];
36  }while(i!=j);
37  cout<<j<<endl;
38 }
39 int main(){
```

```
40    int n,m,p,q,s;
41    cin>>n>>m;
42    //Beolvasás
43    for(int i=0;i<m;i++){
44       cin>>p>>q>>s;
45       G[p].push_back({q,s});
46    }
47    //inicializálás
48    for(int p=1;p<=n;p++){
49       Volt[p]=false;
50       for(int q=1;q<=n;q++){
51          T[p][q]=0;
52       }
53    }
54    nTS=n;
55    for(int p=1;p<=n;p++)
56       if(!Volt[p]) MelyBejar(p);
57    //
58    for (int k=1;k<n;k++)
59       for (int t=1;t+k<=n;t++){
60          int i=TS[t], j=TS[t+k];
61          for(auto q:G[i]){
62             if(iTS[q.pont]>t+k) continue;
63             long ujtav=q.suly+T[q.pont][j];
64             if(ujtav>T[i][j]){
65                T[i][j]=ujtav;
66                Elso[i][j]=q.pont;
67             }
68          }
69       }
70 }
```

Irányított, körmentes súlyozott `G = (V, E, s)` gráf minden `p` pontjára `O(|E|)` időben ki lehet számítani a `p`-ből induló leghosszabb út `Tav[p]` értékét és egy leghosszabb út megadásához szükséges információt.

$$\texttt{Tav[p]} = \begin{cases} 0 & \text{ha Kifok[p]=0} \\ \max\{\texttt{s(p,q)} + \texttt{Tav[q]} : (p,q) \in E\} & \end{cases} \tag{12}$$

A rekurzív összefüggés alapján rekurzió-memorizálás módszerével egyszerűen megvalósítható a kiszámítás.

```
1  #include <bits/stdc++.h>
2  #define maxN 100001
3  using namespace std;
4  struct El{
5    int pont, suly;
6    El(int u, int s){
7      pont=u;
8      suly=s;
9    };
10 };
11 vector<El> G[maxN]; //szomszédsági lista
12 long long Tav[maxN];
13 int Elso[maxN];
14
15 long long TavSzamit(int p){
16   //Globális G
17   if(Tav[p]>=0) return Tav[p];
18   Tav[p]=0;
19   for(auto q:G[p]){
20     long long pqtav=q.suly+TavSzamit(q.pont);
21     if(pqtav>Tav[p]){
22       Tav[p]=pqtav;
23       Elso[p]=q.pont;
24     }
25   }
26   return Tav[p];
27 }
```

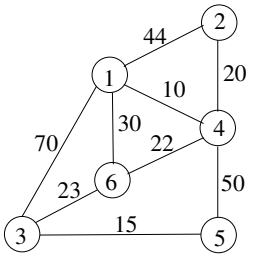
```
35 void UtKiIro(int i){
36   //i~> leghosszabb út kiíratása
37   //Global: Elso
38   do{
39     cout<<i<<" ";
40     i=Elso[i];
41   }while(i!=0);
42 }
43 int main(){
44   int n,m,p,q,s;
45   cin>>n>>m;
46   //Beolvasás
47   for(int i=0;i<m;i++){
48     cin>>p>>q>>s;
49     G[p].push_back({q,s});
50   }
51   //inicializálás
52   for(int p=1;p<=n;p++){
53     Tav[p]=-1;
54     Elso[p]=0;
55   }
56   //
57   for(int p=1;p<=n;p++)
58   TavSzamit(p);
59   UtKiIro(1);
60 }
```

7. Egy pontból induló legrövidebb utak: Dijkstra algoritmus

Bemenet:
`G = (V, E, s)` irányított (irányítatlan) súlyozott gráf és `r` $\in$ `V` kezdőpont, `s : E` $\rightarrow \mathbb{R}^+$
Kimenet:
`Tav : V` $\rightarrow \mathbb{R}^+$, `Tav(p)` $= \delta$`(r, p)`
`FeFa : V` $\rightarrow$ `V`$\bigcup$`{0}`: a legrövidebb utak feszítőfája

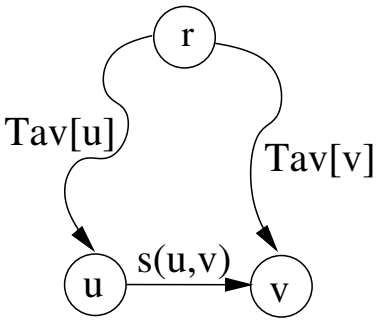
Fokozatos közelítés

Jelölje `Tav[p]` a `p` ponthoz eddig talált legrövidebb út hosszát. Kezdetben a kiibóndulási `r` pontra `Tav[r] = 0`, a többire végtelen.

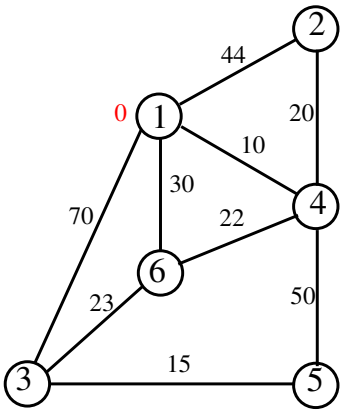


7. ábra

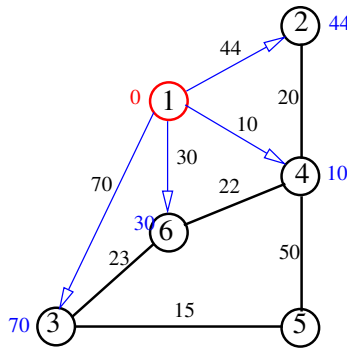
Az algoritmus elve: a legkisebb `Tav` értékű pontra közelítést hajtunk végre.
A legkisebb `Tav` értékű `p` pontra teljesül, hogy `Tav[p]` a legrövidebb `r`-ből `p`-be vezető legrövidebb út hossza.



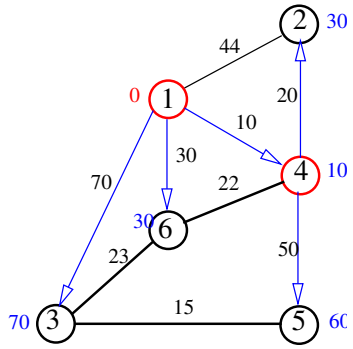
8. ábra. Ha `Tav[u] + s(u, v) < Tav[v]`, akkor `Tav[v] := Tav[u] + s(u, v)` és `Fefa[v] := u`



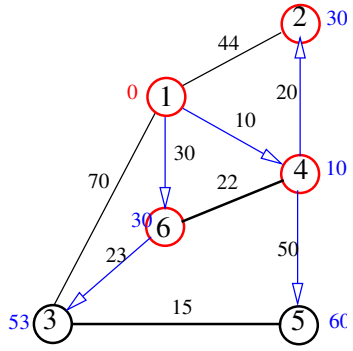
9. ábra. Tav[1]=0, a többi pontra végtelen



10. ábra. Közelítés az 1. pontból



11. ábra. Közelítés az 4. pontból



12. ábra. Közelítés az 6. pontból

Elvi algoritmus

```
Dijkstra(Graf G, int r, Tav, FeFa){
  for(int p=1;p<=n;p++) Tav[p]=Infi;
  Tav[r]=0; FeFa[r]=0;
  S.Sorba(r);
  while (S.size() > 0){
    u=S.Sorbol();
    Kesz[u.pont]=true;
    for(El uv:G[u.pont]){
      if(Kesz[uv.pont]) continue;
      if(Tav[u.pont]+uv.suly<Tav[uv.pont]){//Kozelit(u,v)
        Tav[uv.pont]=Tav[u]+uv.suly;
        FeFa[uv.pont]=u;
        S.Modosit(uv.pont, Tav[uv.pont]);
      }
    }
  }
}
```

Megvalósítás

A használatos fejlesztő szoftver környezetek (pl. C++ STL) a módosítható prioritási sorra nem tartalmaznak közvetlen megvalósítást.
Alapvetően két módszer kínálkozik: `priority_queue` prioritási sor, illetve a `set` rendezett halmaz adattípusok alkalmazásával. `priority_queue` alkalmazása esetén nem tudjuk kivenni a `v` pontot a prioritási sorból, ha `Tav[v]` értéke csökken. Betesszük a `(v, Tav[v])` párt, tehát ugyanazon pont több példányban lehet a prioritási sorban. Amikor kivesszük az `(u, tu)` párt és `Tav[u]` $\neq$ `tu` teljesül, az azt jelenti, hogy `u` már korábban kikerült a helyes `Tav[u] < tu` értékkel, tehát az `(u, tu)` párt el kell dobni.

A `set` adattípus alkalmazásával tudunk adni olyan megvalósítást, amelynek futási ideje $\mathbb{O}$`(m log(n))`, ahol `n` a pontok száma, `m` pedig az élek száma.
Az `S` halmaz elemei a gráf pontjai, a rendezési reláció a `Tav[p]` érték szerint növekvő sorrend. A módosítás miatt bármely `p` pontot meg kell tudni keresni az `S` halmazban, de a `Tav[p]` érték nem azonosítja a `p` pontot. Ezért azonos `Tav` érték esetén az legyen előbb, amely pont sorszáma kisebb. Így a `Tav[p]` és `p` értékek alapján megtalálható `S`-ben az az elem, amely a `p` ponthoz tartozik. És persze `S.begin()` a legkisebb `Tav` értékű lesz `S`-ban.

```
1 //rendezési reláció
2   auto rendez=[&](int x, int y){
3   return Tav[x]<Tav[y] ||Tav[x]==Tav[y] && x<y;
4 };
```

7.1. Megvalósítás `priority_queue` prioritási sorral

```
1  #include <bits/stdc++.h>
2  #define maxN 100001
3  using namespace std;
4
5  struct El{
6    int pont, suly;
7    El(int u, int s){
8      pont=u; suly=s;
9    };
10   bool operator<(const El& jobb) const{
11     return suly>jobb.suly || (suly==jobb.suly && pont>jobb.pont);
12   }
13 };
14 typedef
15 vector<El> Graf[maxN];//éllista
16 Graf G;
17 int A,B;
18
19 int  Beolvas(){
20   int n, m,u,v,s;
21   cin>>n>>m;
22   cin>>A>>B;
23   for(int i=0; i<m; i++){
24     cin>>u>>v>>s;
25     G[u].push_back(El(v,s));
26     G[v].push_back(El(u,s));
27   }
28   return n;
29 }
```

```
int FeFa[maxN];
int Tav[maxN];

void Dijkstra(Graf G, int n, int r, int Tav[], int FeFa[]){
  priority_queue<El> S;
  int Infi=INT_MAX/2;
  for (int v=1; v<=n; ++v){//inicializálás
    Tav[v]=Infi; FeFa[v]=-1;
  }
  Tav[r]=0; FeFa[r]=0;
  S.push(El(r,0));
  int ujtav;
  while (S.size() > 0){
    auto [u, uTav]=S.top(); S.pop();
    if(Tav[u]!=uTav) continue;//Tav[u] már kész
    for(auto [v, uvsuly] : G[u]){
      ujtav=uTav+uvsuly;
      if (ujtav<Tav[v]){//u->v közelítés
        S.push({v,Tav[v]=ujtav});
        FeFa[v]=u;
      }
    }
  }
}
```

```
int main(){
  int n=Beolvas();

  Dijkstra(G, n, A, Tav, FeFa);

  if(FeFa[B]<0){cout<<"Nincs út"; return 0;}
  int Ut[maxN];
  int x=B;
  int m=0;
  while (x!=A){
    Ut[m++]=x;
    x=FeFa[x];
  }
  Ut[m]=A;
  cout<<Tav[B]<<endl;
  for (int i=m; i>=0; i--)
  cout<<Ut[i]<<" ";
  cout<<endl;
}
```

7.2. Megvalósítás `set` rendezett halmazzal

```
1  void Dijkstra(Graf G, int n, int r, int Tav[], int FeFa[]){
2    //Megvalósítás set adattípussal
3    int Infi=INT_MAX/2;
4    //rendezési reláció a prioritási sorhoz:
5    auto rendez=[&](int x, int y){
6      return Tav[x]<Tav[y] || Tav[x]==Tav[y] && x<y;
7    };
8    auto S=set<int, decltype(rendez)>(rendez);
9    for (int v=1; v<=n; ++v){//inicializálás
10     Tav[v]=Infi; FeFa[v]=-1;
11   }
12   Tav[r]=0; FeFa[r]=0;
13   S.insert(r);
14   int ujtav;
15   while (S.size() > 0){
16     auto us=S.begin();//legkisebb Tav-értékű S-ben
17     int u=*us;
18     S.erase(us);//kivesszük a prioritási sorból
19     for(auto [v, uvsuly]:G[u]){//minden szomszédra
20       ujtav=Tav[u]+uvsuly;
21       if (ujtav<Tav[v]){//u->v közelítés
22         if(Tav[v]<Infi) //ha már a sorban van v
23         S.erase(v);    //kivesszük v-t
24         Tav[v]=ujtav;
25         FeFa[v]=u;
26         S.insert(v);
27       }
28     }
29   }
30 }
```

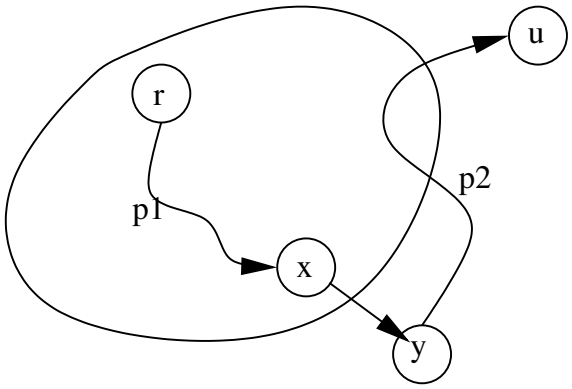
7.3. A Dijkstra algoritmus helyessége

A korábbiakban bevezetett jelölés szerint minden $u, v \in V$ pontpárra $\delta(u, v)$ a legrövidebb $u \rightsquigarrow v$ út hossza.

7.1. Állítás. $T[u] \geq \delta(r, u)$ *Az állítás igaz, mert minden él súlya* ≥ 0.

7.2. Állítás. *Amikor* u *kikerül az* S *halmazból, akkor* $T[u] \leq \delta(r, u)$.

Bizonyítás indirekt módon.
Tegyük fel, hogy u az első olyan pont ami kikerül S-ból és $T[u] \neq \delta(r, u)$.



13. ábra

Legyen $P : r \rightsquigarrow x \rightarrow y \rightarrow u$ egy legrövidebb út r-ből u-ba, és legyen x a P úton az első olyan pont, amely már nincs az S halmazban, y pedig a következő pont az úton, tehát $(x, y) \in E$. Jelölje $p1$, és $p2$ a P legrövidebb útnak az ábrán megadott két részútját.

Tekintsük a következő négy észrevételt:
(1) $T[x] = \delta(r, x)$
Igaz, mert x már kikerült az S-ből és u az első, amelyre nem teljesül az egyenlőség.

(2) $T[y] = T[x] + s(x, y)$
Igaz, mert amikor x kikerült az S prioritási sorból, akkor végrehajtódott közelítés az $x \rightarrow y$ élre, tehát ha $T[y] > T[x] + s(x, y)$ lett volna, akkor $T[y] = T[x] + s(x, y)$ lett.

(3) $T[u] \leq T[y]$
Igaz, mert u előbb kerül ki az S-ből, ami $T[]$ szerinti minimumos prioritási sor.

(4) $\delta(r, y) = \delta(r, x) + s(x, y)$ és $\delta(r, u) = \delta(r, y) + \delta(y, u)$
Igaz, mert P legrövidebb $r \rightsquigarrow u$ út.

Az (1) -(4) észrevételeket felhasználva kapjuk, hogy:

$$T[u] \leq T[y] \leq T[x] + s(x, y) = \delta(r, y) \leq \delta(r, y) + \delta(y, u) = \delta(r, u)$$

, ami ellentmond e feltevésünknek, hogy $T[u] > \delta(r, u)$.

8. Feladat1: Hosszú út; Mester/Legrövidebb utak 39

A Globus utazási iroda autóbuszos túrákat szervez. Minden út adott U városból adott V városba vezet esetleg több különböző városon keresztül. Az útvonalon két város között nem áll meg a busz, de minden városban megáll. Az útvonal kiválasztásának elsődleges szempontja, hogy az összesen megtett út hossza a lehető legkisebb legyen. Másodlagos

szempont, hogy a buszvezető-nek mennyit kell folyamatosan vezetnie. Ezért, ha több legrövidebb út is lehetséges, ezek közül azt kell választani, amelyen a két szomszédos város közötti távolságok maximuma a lehető legkisebb legyen. Írj programot, amely megadott várospárokra kiszámítja a legalkalmasabb útvonalat!

Bemenet

A `standard bemenet` első sorában három egész szám van, a városok `N` száma $(2 \leq N \leq 500)$, a közvetlen utal `M` $(2 \leq M \leq 10\,000)$ és a kérdések `K` $(2 \leq K \leq 1\,000)$ száma. A következő `M` sor mindegyike egy `u v t` számhármast tartalmaz $(1 \leq u \neq v \leq N,\ 1 \leq t \leq 1000)$ amely azt jelenti, hogy az u és v város között t hosszú útszakasz van. Bármely két város között legfeljebb egy útszakasz van. A közvetlen utak kétirányúak.

Kimenet

A `standard kimenetre` `2K` sort kell kiírni, minden kérdése két sort. Az első sor az T R számpárt tartalmazza, ahol T az útvonal hossza, R pedig az útvonalban két szomszédos város közötti legnagyobb távolság. A második sor tartalmazza az útvonalat. Ha nincs út a kérdésben szereplő két város között, akkor az első sorba a 0 0 számpárt kell írni, a második sor pedig legyen üres sor! Több megoldás esetén bármelyik megadható.

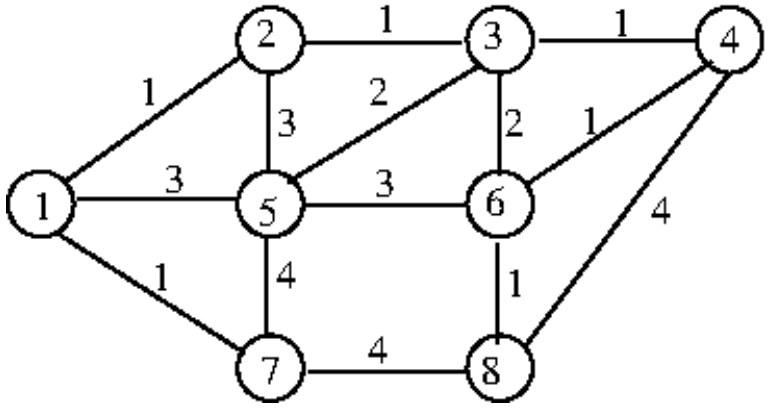
Példa

Bemenet

```
8 14 3
1 2 1
1 5 3
1 7 1
2 3 1
2 5 3
3 4 1
3 5 2
3 6 2
4 6 1
4 8 4
5 6 3
5 7 4
6 8 1
7 8 4
1 8
5 6
2 6
```

Kimenet

```
5 1
1 2 3 4 6 8
3 3
5 6
3 1
2 3 4 6
```



Korlátok

Időlimit: 0.8 mp.
Memórialimit: 64MiB

Pontozás

A pontszámok 50%-a jár, ha helyes a kiírandó első sor.
A pontok 10%-a kapható olyan tesztekre, ahol `N < 16`.
A pontok 20%-a kapható olyan tesztekre, ahol `N` $\leq$ `100`.
A pontok 70%-a kapható olyan tesztekre, ahol nincs egyéb korlátozás.

9. Feladat2: Leggyorsabb szállítás; Mester/Legrövidebb utak 25

Ismerjük egy ország vasútvonalairól az egyes várospárok közötti pályaszakaszok hosszát, vala-mint teherbírását. Bármely két város között csak egyetlen pályaszakasz van.

Készíts programot, amely megadja két városra a közöttük levő legrövidebb utat, több lehetséges útvonal esetén a legnagyobb teherbírásút!

Bemenet

A `standard bemenet` első sorában a városok száma $(1 \leq N \leq 10\,000)$ és a pályaszakaszok száma $(1 \leq M \leq 100\,000)$ van. A következő M sorban egy-egy pályaszakasz két végpontjának sorszáma $(1 \leq Ai \neq Bi \leq N)$, a pálya-szakasz hossza $(1 \leq Hi \leq 10\,000)$ és teherbírása $(1 \leq Ti \leq 10\,000)$ van. Az utolsó sorban a két város sorszáma szerepel $(1 \leq V \neq W \leq N)$. A `V` városból a `W` városba biztosan el lehet jutni.

Kimenet

A `standard kimenetre` első sorába a két város közötti legrövidebb út hosszát és ennek maximális teherbírását kell kiírni! A második sorba egy olyan legrövidebb út pontjai sorszámai kerüljenek, amelyen a legkisebb teherbírású pályaszakasz a lehető legnagyobb! A második sor első sorszáma `V`, az utolsó pedig `W` legyen! Ha több ilyen út is lenne, tetszőleges kiírható.

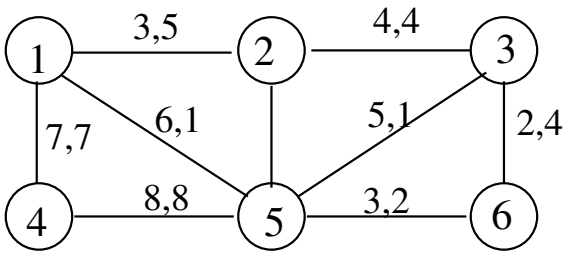
Példa

Bemenet

```
6 9
1 2 3 5
1 4 7 7
1 5 6 1
2 3 4 4
2 5 3 5
4 5 8 8
3 5 5 1
3 6 2 4
5 6 3 2
1 6
```

Kimenet

```
9 4
1 2 3 6
```



Korlátok

Időlimit: 0.1 mp.
Memórialimit: 32MiB

9.1. Gyakorló feladatok

https://cses.fi/problemset/task/1195 Flight Discount
https://cses.fi/problemset/task/1196 Flight Routes